



Propuesta de arquitectura

Plataforma global de venta de boletos para eventos en vivo

System Design Challenge

Autor	Rolando Palermo Rodríguez Cruz
Fecha	06 de septiembre de 2026
Versión	1.0
Clasificación	Propuesta técnica

Control de cambios

Autor	Fecha	Descripción
Rolando Palermo Rodríguez Cruz	06/09/2026	Versión inicial de la propuesta: arquitectura global, decisiones técnicas e infraestructura como código / CI/CD.

Índice

Control de cambios	1
1. Diagrama de arquitectura	2
1.1. Contexto y alcance	2
1.2. Arquitectura lógica	3
1.3. Flujos críticos	3
1.3.1. Flash crowd y control de admisión	3
1.3.2. Concurrencia de inventario	5
1.3.3. Pago, boleto y conciliación	7
2. Decisiones importantes	7
2.1. Inventario como fuente de verdad	7
2.2. Picos de tráfico	7
2.3. Consistencia de pago	8
2.4. Transactional outbox e idempotencia	9
2.5. Multi-región y resiliencia	9
2.6. Observabilidad	9
2.7. Tradeoffs principales	9
3. Infraestructura como código & CI/CD	10
3.1. Estructura de Terraform	10
3.2. Versionado y cambios	11
3.3. Secretos	11
3.4. Pipeline CI/CD	11
Conclusión	11

1. Diagrama de arquitectura

La arquitectura está pensada para operar en cuatro países, absorber picos de 10 a 20 veces el tráfico habitual y proteger dos invariantes: no vender dos veces el mismo asiento y no capturar un pago sin boleto.

1.1. Contexto y alcance

LiveTix opera en México, Estados Unidos, España y Japón. La propuesta usa tres regiones de AWS:

Región AWS	Mercados
us-east-1	México y Estados Unidos
eu-west-1	España
ap-northeast-1	Japón

- La capa global *Route 53* + *CloudFront* + *WAF* es única.
- Los servicios de aplicación y los datos transaccionales se despliegan por región.
- Cada evento tiene una *home region*.
- Las lecturas no críticas (catálogo de eventos, información del evento, imágenes/banners, recomendaciones, etc.) pueden resolverse cerca del usuario.
- La reserva y la venta de asientos siempre se escriben en la región propietaria del evento.

1.2. Arquitectura lógica

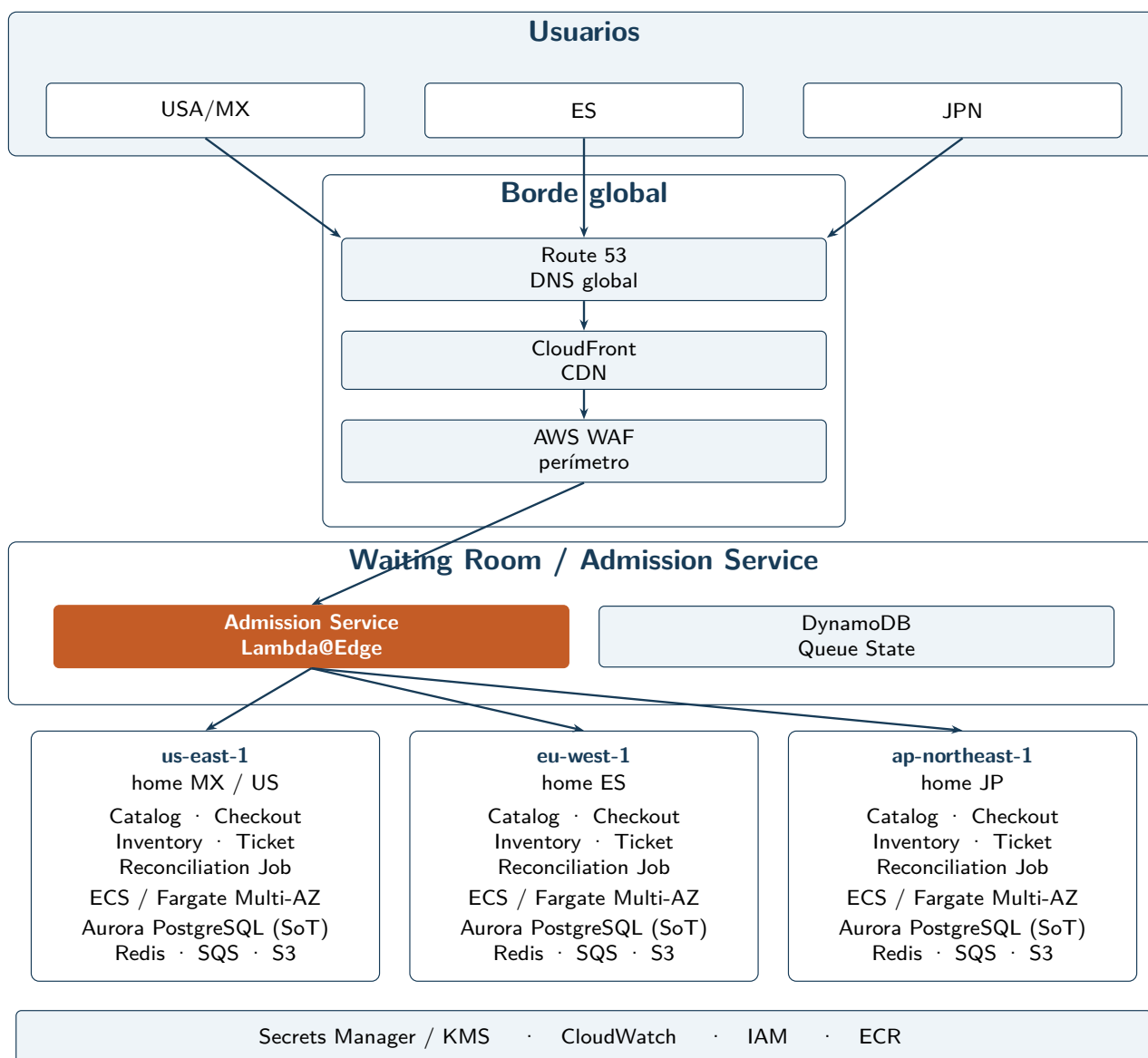


Figura 1: Arquitectura global de LiveTix: usuarios entran por Route 53 hacia CloudFront y WAF; Waiting Room en Lambda@Edge con DynamoDB; tres regiones con fuente de verdad local y job de reconciliación.

1.3. Flujos críticos

1.3.1. Flash crowd y control de admisión

Cuando se liberan boletos, el sistema no espera a que ECS reaccione. En su lugar:

- Route 53 resuelve el DNS; CloudFront y WAF absorben y filtran.
- La sala de espera (Admission Service en Lambda@Edge) dosifica el ingreso al checkout; DynamoDB guarda el estado de la cola.
- La capacidad de Fargate se preescala antes del evento.

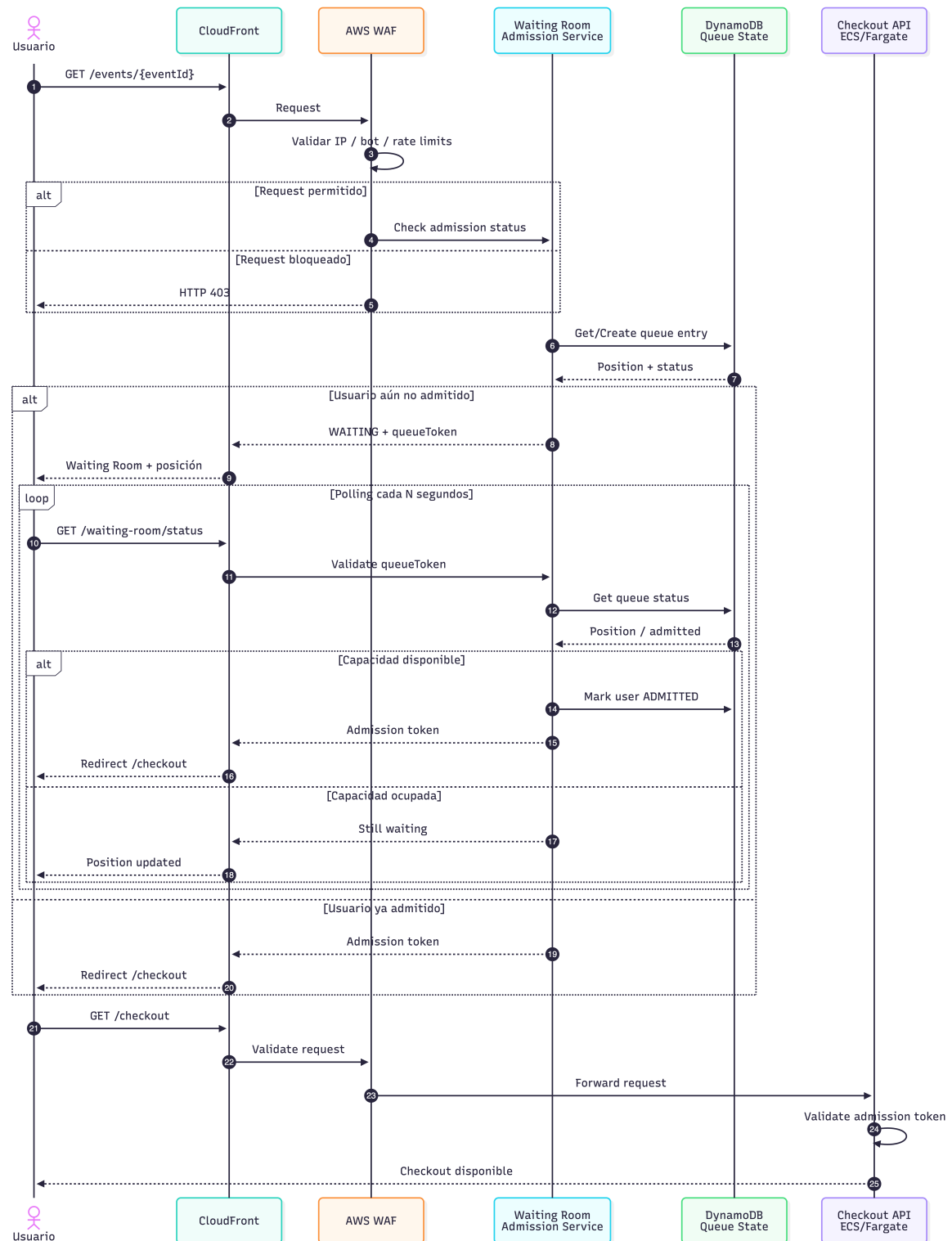


Figura 2: Flujo de admisión al checkout. CloudFront y WAF filtran; Admission Service y DynamoDB dosifican el ingreso antes de Fargate.

1.3.2. Concurrencia de inventario

La reserva es una actualización condicional en Aurora. El asiento pasa de **AVAILABLE** a **HELD** y después a **SOLD**, o regresa a **AVAILABLE** si el hold expira.

- El `hold_id` es el token de propiedad temporal.
- El worker de expiración es idempotente: no puede liberar un hold nuevo que ya substituyó al anterior.

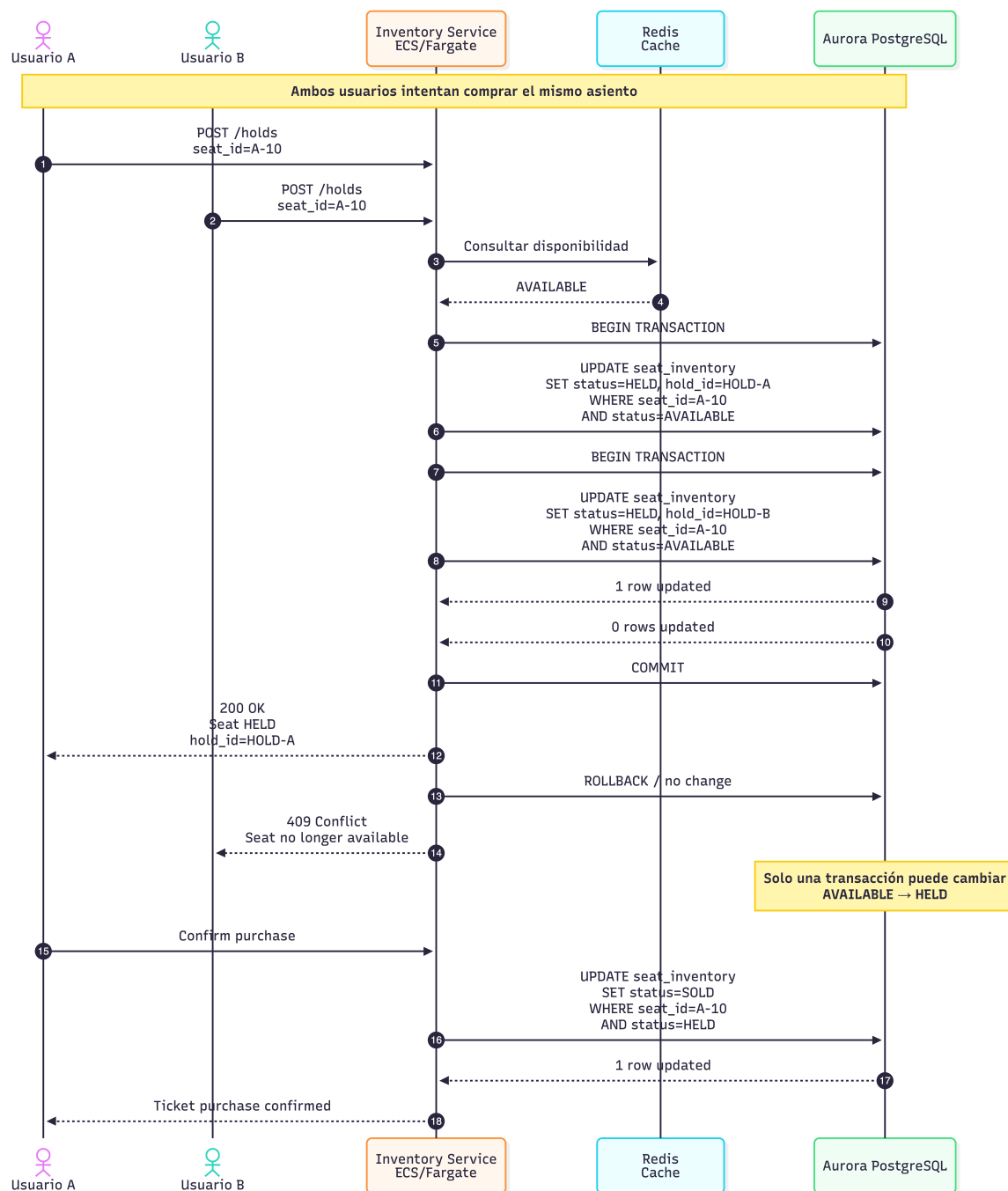


Figura 3: Concurrency de inventario. Dos compradores intentan el mismo asiento; Aurora confirma un solo hold.

1.3.3. Pago, boleto y conciliación

El hold del asiento queda en el flujo de inventario. La autorización, la emisión del boleto, la captura y la conciliación se detallan en la sección de consistencia de pago (Figura 4).

2. Decisiones importantes

Las decisiones privilegian consistencia, absorción de picos y una operación que un equipo de cinco ingenieros pueda sostener.

2.1. Inventario como fuente de verdad

Aurora PostgreSQL es la autoridad del asiento. El inventario numerado exige exclusión mutua. Una actualización condicional (`status = AVAILABLE`) más una restricción de unicidad evitan la doble venta.

Reserva atómica. Solo actualiza el asiento si sigue `AVAILABLE`:

```
UPDATE seat_inventory
SET status = 'HELD',
    hold_id = :hold_id,
    hold_expires_at = now() + interval '5 minutes'
WHERE event_id = :event_id
    AND seat_id = :seat_id
    AND status = 'AVAILABLE';
```

Liberación de asiento. El worker solo libera el hold que le corresponde y que ya venció:

```
UPDATE seat_inventory
SET status = 'AVAILABLE',
    hold_id = NULL,
    hold_expires_at = NULL
WHERE event_id = :event_id
    AND seat_id = :seat_id
    AND status = 'HELD'
    AND hold_id = :hold_id
    AND hold_expires_at <= NOW();
```

Si el `UPDATE` de reserva afecta 0 filas, otro comprador ya tomó el asiento. La condición `hold_id = :hold_id` en la liberación hace que un worker retrasado no deshaga un hold posterior.

2.2. Picos de tráfico

El autoescalado reactivo llega tarde cuando la demanda crece en segundos. La defensa es en capas:

- Preescalado de Fargate antes de eventos anunciados.
- CloudFront para lo estático y cacheable.
- WAF en el perímetro.
- Waiting Room para dosificar el checkout.

- SQS para trabajo asíncrono que no debe competir con la reserva.

El objetivo no es que todos lleguen al mismo tiempo al asiento, sino que quienes pasan el cupo encuentren un sistema estable.

2.3. Consistencia de pago

La pasarela es dueña del instrumento; LiveTix es dueño de la orden, el asiento y el boleto. La integración usa claves de idempotencia. Autorizar y capturar van separados a propósito: no se cobra en firme si todavía no hay boleto. El flujo, a partir de una orden ya en hold, se muestra en la Figura 4.

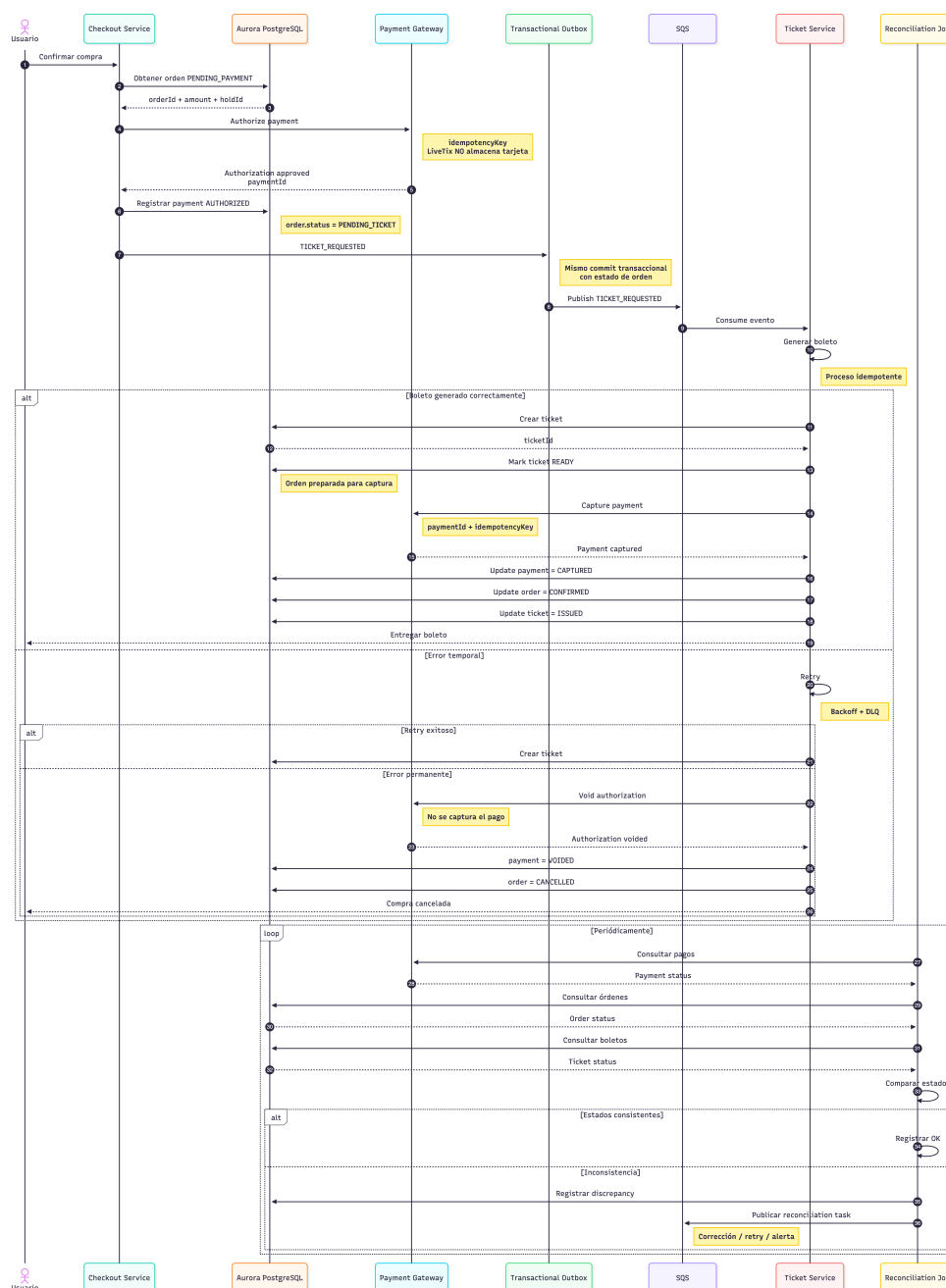


Figura 4: Pago, boleto y conciliación. La captura ocurre después de emitir el boleto; si la emisión falla de forma permanente se hace *void* y un reconciliador cierra huecos entre sistemas.

2.4. Transactional outbox e idempotencia

El evento de emisión se persiste en la misma transacción que confirma asiento y orden. Un publisher aparte lo coloca en SQS. Los consumidores asumen entrega *at-least-once*: la generación del boleto y la captura deben ser idempotentes.

2.5. Multi-región y resiliencia

Cada región corre en varias Availability Zones. La escritura crítica no cruza de región en el camino feliz. Ante una caída regional completa se recupera en una región secundaria; RPO y RTO se confirman con pruebas de failover, no solo con el diseño.

2.6. Observabilidad

Alarmas prioritarias:

- Tasa de éxito de checkout.
- Conflictos de reserva de inventario.
- Fallos de autorización y de captura.
- Latencia y errores de emisión de boletos.
- Profundidad y antigüedad de SQS.
- Órdenes sin boleto.
- Pagos sin boleto correspondiente.
- Errores por región y por Availability Zone.

La métrica de negocio que no puede quedar en verde con mentiras es `payment_ticket_inconsistency = 0`.

2.7. Tradeoffs principales

Decisión	Beneficio	Costo
<i>Home region</i> por evento	Consistencia fuerte del inventario	Una compra puede cruzar región
Aurora como fuente de verdad	Evita double-selling	Más costo que un cache puro
Waiting Room	Protege el checkout en picos	El usuario espera
SQS + outbox	Absorbe carga y sobrevive a fallos parciales	Más piezas que operar
Fargate	Menos servidores que administrar	Menos control fino que EC2
Multi-AZ + DR regional	Resiliencia real	Costo y ejercicios de failover
Sin K8s ni service mesh	Operable por un equipo pequeño	Menos portabilidad multi-cloud

3. Infraestructura como código & CI/CD

Terraform y el pipeline tienen que permitir levantar ambientes de forma repetible, aislar las regiones y hacer despliegues sin que terminemos manteniendo una plataforma paralela que se vuelva demasiado pesada para un equipo de cinco personas. El state de Terraform se mantiene en un bucket S3 dedicado, con versionado, cifrado y acceso controlado por IAM, para evitar perderlo o que alguien lo modifique directamente.

3.1. Estructura de Terraform

```
infrastructure/
|
+-- modules/
|   +-- networking/
|   +-- ecs/
|   +-- aurora/
|   +-- redis/
|   +-- sqs/
|   +-- dynamodb/
|   +-- waf/
|   +-- waiting-room/
|   +-- secrets/
|   +-- observability/
|
+-- environments/
|   +-- dev/
|   +-- staging/
|   +-- production/
|       +-- us-east-1/
|       +-- eu-west-1/
|       +-- ap-northeast-1/
|
+-- global/
    +-- route53/
    +-- cloudfront/
    +-- iam/
```

- Los módulos expresan el patrón reutilizable (servicio ECS, cluster Aurora, cola, tabla DynamoDB, WAF, Waiting Room, secretos, observabilidad).
- CloudFront vive solo en `global/`. El Waiting Room es Lambda@Edge asociado a esa distribución; WAF se adjunta al mismo borde.
- Los environments solo pasan capacidad, red y flags, e instancian `modules/secrets` por ambiente.
- Producción se parte por región para limitar el blast radius.
- Los recursos globales (Route 53, CloudFront y IAM) viven fuera de los stacks regionales.

3.2. Versionado y cambios

Versiones de módulos y providers van fijadas. Todo cambio entra por pull request, con **terraform plan** en CI y apply solo después de revisión.

3.3. Secretos

Nada de secretos en git ni en **tfvars** versionados. Secrets Manager y KMS, con IAM de mínimo privilegio y roles de tarea en lugar de access keys estáticas.

3.4. Pipeline CI/CD

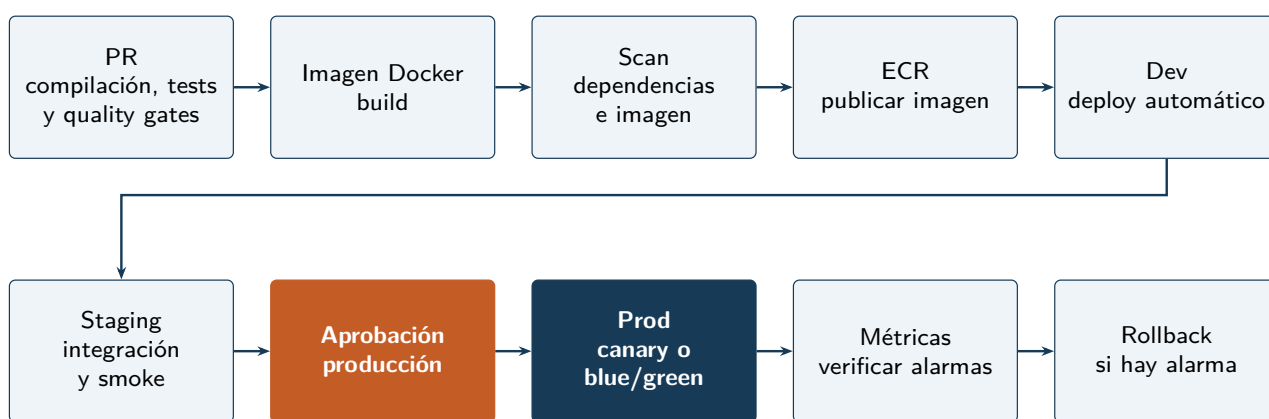


Figura 5: Pipeline: verificación automática hasta staging; producción con aprobación, canary y rollback por alarmas.

Infraestructura sigue un pipeline paralelo: **fmt**, **validate**, **plan** en el PR y **apply** controlado por ambiente. Las migraciones de base de datos deben ser compatibles con el código anterior para no romper un canary a medias.

Conclusión

1. La propuesta no maximiza sofisticación. Busca tres garantías: el checkout no colapsa en un flash crowd, un asiento se vende una sola vez y una captura de pago tiene boleto.
2. Outbox, consumidores idempotentes y reconciliación cubren los huecos entre LiveTix y la pasarela.
3. Terraform, ambientes aislados y un pipeline con canary permiten operar esto con un equipo pequeño.